

MULTI-AGENT RESEARCH PROJECT

Research-Mind

InsightForge

Project Explanation Guide

Interview ke liye sirf zaroori coverage

Search Agent → Reader Agent → Writer Chain → Critic Chain

OpenAlex + Google News + optional Tavily
Gemini research + optional Groq critic
Streamlit UI + Docker/Cloud deployment

Goal: project ko clearly explain karna, code ko defend karna, aur honest limitations batana.

Contents

1 Project Ko 60 Seconds Mein Kaise Explain Karein	2
1.1 Hinglish pitch	2
1.2 English pitch	2
1.3 Problem and users	2
2 Original Tutorial Se Final Project Tak	3
3 Architecture and Complete Flow	3
3.1 High-level architecture	3
3.2 Button click ke baad exact sequence	3
4 File-by-File Explanation	4
4.1 agents.py	4
4.2 tools.py	4
4.3 pipeline.py	5
4.4 app.py	5
5 Four Stages Ko Detail Mein Explain Karein	6
5.1 1. Search Agent	6
5.2 2. Reader Agent	6
5.3 3. Writer Chain	6
5.4 4. Critic Chain	6
6 Sources, Citations and Hallucination Control	6
7 Important Decisions and Problems Solved	7
7.1 Why Gemini + Groq?	7
7.2 Groq 413 request-too-large error	7
7.3 PEP 668 installation error	7
7.4 Secrets and deployment	8
8 Most Likely Interview Questions	8
9 Two-Minute Demo and Final Revision	9
9.1 Demo order	9
9.2 Five lines yaad honi chahiye	9
9.3 Final checklist	10

1 Project Ko 60 Seconds Mein Kaise Explain Karein

One-line definition

Research-Mind ek evidence-based research application hai jo recent papers aur news search karta hai, sources ko compare karta hai, cited report likhta hai, aur final report ko Critic se review karwata hai.

1.1 Hinglish pitch

“Maine tutorial ke Search Agent, Reader Agent, Writer Chain aur Critic Chain ko base rakha. Search stage OpenAlex, Google News aur optional Tavily se live sources laati hai. Reader un sources ko compare karta hai, Writer source IDs ke saath report banata hai, aur Groq Critic weak claims check karta hai. Gemini main research model hai. Maine iske upar responsive Streamlit UI, report export, follow-up Q&A aur deployment support add kiya.”

1.2 English pitch

“I upgraded a tutorial-based Search-Reader-Writer-Critic workflow into a usable research application. It retrieves recent scholarly and news evidence, assigns traceable source IDs, compares findings, writes a cited report, and runs an optional independent Groq review. Gemini handles the main research stages, while Streamlit provides the interface, export, and follow-up experience.”

1.3 Problem and users

Search engine links deta hai, jabki normal chatbot fluent answer deta hai lekin source grounding weak ho sakti hai. Project dono ko combine karta hai.

Target users:

- student jo kisi topic ka reliable overview chahta hai;
- researcher jo latest papers aur signals compare karna chahta hai;
- founder/policy professional jo trends aur limitations dono dekhna chahta hai;
- koi bhi person jo source-linked research brief chahta hai.

2 Original Tutorial Se Final Project Tak

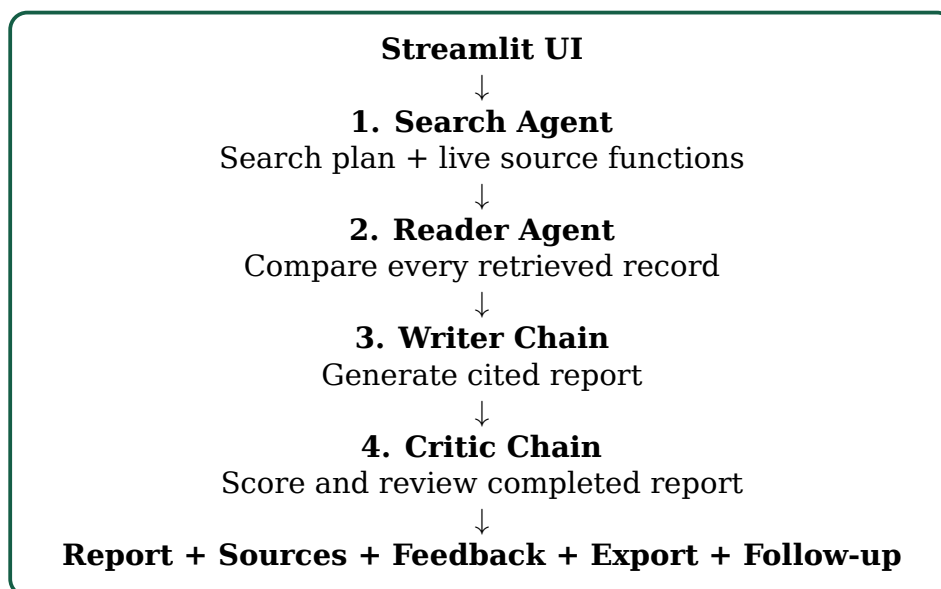
Stage	Original idea	Current improvement
Search Agent	Tavily web search	Search plan + OpenAlex + Google News + optional Tavily
Reader Agent	Ek selected URL scrape	Saare retrieved abstracts/snippets compare
Writer Chain	Generic report	Source-ID citations, uncertainty and audience-aware report
Critic Chain	Score and feedback	Source support, bias, missing evidence and overconfidence check
Interface	Basic Streamlit/demo output	Responsive form, progress, tabs, source cards, download, follow-up
Models	One OpenAI model	Gemini research + optional Groq critic

Interview mein honest line

Maine tutorial ka mental model replace nahi kiya. Maine same four stages ko better sources, grounded prompts aur proper UI ke saath improve kiya. Code ko deliberately sequential aur simple rakha hai.

3 Architecture and Complete Flow

3.1 High-level architecture



3.2 Button click ke baad exact sequence

1. User question, time range, depth, language and sources select karta hai.
2. app.py values ko `run_research_pipeline()` mein pass karta hai.

3. Pipeline Gemini main model aur optional Groq critic banati hai.
4. Search Agent question ka short research plan banata hai.
5. Question se common words remove karke search query banti hai.
6. Selected source functions one-by-one call hote hain.
7. Duplicate sources remove hote hain aur S1, S2 IDs milti hain.
8. Reader Agent records compare karke structured notes banata hai.
9. Writer Chain notes + exact sources se cited report banati hai.
10. Critic Chain completed report ko sources ke against review karti hai.
11. Streamlit report, notes, sources, feedback and search plan tabs mein dikhata hai.

Why sequential? Har stage ko previous stage ka output chahiye. Sequential flow beginner-friendly, traceable aur interview mein easy to draw hai.

4 File-by-File Explanation

4.1 agents.py

Is file mein LLM-related code hai:

- `create_models()` Gemini aur optional Groq banata hai;
- four `ChatPromptTemplate` objects roles define karte hain;
- builder functions `prompt | model | StrOutputParser()` return karte hain;
- `answer_follow_up()` collected context ke andar answer deta hai.

Listing 1: Simple model rule

```
main_model = ChatGoogleGenerativeAI(...)

if GROQ_API_KEY:
    critic_model = ChatGroq(...)
else:
    critic_model = main_model
```

Google key required hai. Groq optional hai. Koi automatic router ya multiple fallback chain nahi hai.

4.2 tools.py

Yeh external data laata hai; LLM imagination par retrieval depend nahi karti.

Function/source	Responsibility
<code>search_openalex()</code>	Recent scholarly papers, authors, venue, DOI, abstract and cited-by count
<code>search_news()</code>	Google News RSS se recent reporting
<code>search_tavily()</code>	Optional general-web results when key exists
<code>collect_evidence()</code>	Selected sources run, errors warn, duplicates remove, IDs assign
<code>format_evidence()</code>	Records ko model-readable text banata hai
<code>web_search()</code>	Original tutorial-compatible wrapper
<code>scrape_url()</code>	Original tutorial-compatible bounded URL reader

Each source call ka timeout hai. Ek source fail ho to warning milti hai aur next source continue hota hai.

4.3 pipeline.py

Yeh project ka easiest file to explain hona chahiye. Isme:

- normal function arguments user configuration receive karte hain;
- ek plain state dictionary intermediate output store karti hai;
- four stages visible order mein call hote hain;
- optional callback UI progress update karta hai;
- `export_markdown()` downloadable report banata hai.

Important state:

```
state = {
    "request": {...},
    "models": {"research": "Gemini", "critic": "Groq"},
    "search_plan": "...",
    "search_results": "...",
    "evidence": [...],
    "scraped_content": "...", # Reader notes
    "report": "...",
    "feedback": "..."
}
```

4.4 app.py

Streamlit UI responsibilities:

- local `.env` aur Streamlit Cloud secrets support;
- question form and optional settings;
- four-stage progress display;
- report/notes/sources/feedback/search-plan tabs;
- source cards and original links;
- Markdown download;
- result and follow-ups ko `st.session_state` mein preserve karna.

5 Four Stages Ko Detail Mein Explain Karein

5.1 1. Search Agent

LLM pehle question ko scope karta hai: kya search karna hai, sub-questions kya hain aur blind spots kya ho sakte hain. Uske baad pipeline deterministic source functions call karti hai.

Important honesty: current Search Agent unrestricted autonomous tool-calling agent nahi hai. Yeh controlled prompt chain + explicit search functions ka stage hai. Isse behaviour predictable rehta hai.

5.2 2. Reader Agent

Original Reader ek URL read karta tha. Current Reader multiple records compare karta hai. Output mein important findings, emerging signals, disagreements aur missing evidence hota hai. Source text ko data treat karne aur embedded instructions ignore karne ka prompt hai.

5.3 3. Writer Chain

Reader notes aur exact source records receive karke report banati hai. Prompt ke main rules:

- factual claims ke paas [Sx] citation;
- source/statistic invent nahi karna;
- weak evidence par uncertainty show karna;
- requested language and audience follow karna.

5.4 4. Critic Chain

Completed report aur sources receive karti hai. Score, strengths, improvements and verdict return karti hai. Groq available ho to independent model review milta hai; otherwise Gemini review karta hai.

Current Critic feedback report ko automatically rewrite nahi karti. Yeh visible review hai. Revision loop future improvement hai.

6 Sources, Citations and Hallucination Control

Har evidence record roughly yeh structure rakhta hai:

```
{
  "id": "S1",
  "kind": "Academic",
  "title": "...",
  "url": "https://...",
  "published": "2026-...",
  "publisher": "...",
  "summary": "..."
}
```

Hallucination reduce karne ke methods:

- live sources model context mein diye jate hain;
- report sirf existing [Sx] IDs cite karne ko kehti hai;
- temperature 0.15 rakha hai;
- snippets bounded hain, so context predictable hai;
- Critic unsupported/overconfident claims check karta hai;
- UI original source link show karti hai.

Honest limitation

Source ID valid hona yeh guarantee nahi karta ki source claim ko logically support bhi karta hai. Automated claim-citation entailment checker abhi nahi hai. High-stakes claims human ko original source se verify karne chahiye.

Isse vector-database RAG mat bolo. Better term: **live retrieval-augmented research pipeline**, kyunki APIs/RSS se evidence run-time par retrieve hota hai; embeddings/vector store use nahi hote.

7 Important Decisions and Problems Solved

7.1 Why Gemini + Groq?

Gemini main research flow handle karta hai. Groq optional independent Critic hai. Fixed rule automatic router se easier to understand hai. Second model correctness guarantee nahi karta, but different review perspective deta hai.

7.2 Groq 413 request-too-large error

Problem: report + evidence + output budget provider limit cross kar rahe the.

Simple fix:

- per-source summaries 500 characters tak;
- Quick/Standard/Deep source limits 4/6/8;
- Reader, Writer and Critic inputs par direct string limits;
- maximum output tokens environment variable se controlled.

7.3 PEP 668 installation error

System Python par pip install block hua. Correct solution project virtual environment tha:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
streamlit run app.py
```

7.4 Secrets and deployment

Local development .env use karti hai. Streamlit Cloud TOML secrets use karta hai. Required/optional configuration:

```
GOOGLE_API_KEY = "..." # required
GOOGLE_MODEL = "gemini-3.6-flash"
GROQ_API_KEY = "..." # optional critic
GROQ_MODEL = "openai/gpt-oss-20b"
TAVILY_API_KEY = "..." # optional web search
MAX_OUTPUT_TOKENS = 1400
```

Real keys Git mein commit nahi honi chahiye. Exposed key ko revoke/rotate karna chahiye.

8 Most Likely Interview Questions

Q1. Why multi-agent instead of one prompt?

Search planning, evidence reading, report writing and quality review different responsibilities hain. Four outputs inspect kiye ja sakte hain, aur flow tutorial se naturally map hota hai. Trade-off: extra latency and model calls.

Q2. Are these fully autonomous agents?

No. They are bounded role-based LLM stages in a controlled pipeline. Search tools explicitly Python pipeline call karti hai. I chose predictability and explainability over unrestricted autonomy.

Q3. Why does Reader use snippets instead of full articles?

Full scraping introduces paywalls, robots rules, copyright, parsing and latency issues. Abstracts and snippets are a safe baseline. Future version official full-text/open-access APIs use kar sakti hai.

Q4. Why OpenAlex?

It provides scholarly metadata, dates, authors, DOI links, venues, cited-by counts and many abstracts without requiring an API key.

Q5. Why is Tavily optional?

Academic and news research should work without a paid search key. Tavily adds broader web coverage only when configured.

Q6. How do citations work?

Collector real records ko S1, S2 IDs deta hai. Writer receives those exact records and uses IDs inline. UI/export same IDs ko source URLs se map karte hain.

Q7. What if a source fails?

Each source call has simple error handling. Failure warning ban jati hai and remaining selected sources continue.

Q8. Why are calls sequential?

This is a learning and interview project with only a few sources. Sequential code is easier to trace and debug. Parallel I/O can be added later if measured latency requires it.

Q9. Why Gemini for research and Groq for Critic?

Gemini is the fixed main model and Groq gives an optional independent review. The rule is explicit and simple; I removed unnecessary automatic routing.

Q10. Biggest limitation?

The system relies mainly on metadata/snippets and does not automatically verify that every citation entails its claim. Evidence quality and deterministic citation checking are the first improvements.

Q11. Why Streamlit?

It lets a Python project ship forms, progress, tabs, chat and downloads quickly. For a larger multi-user product I would separate a FastAPI backend and React frontend.

Q12. How would you scale it?

Add background jobs, persistent research sessions, caching, rate limits, authentication and async/parallel source requests. I would add these only when real usage justifies the complexity.

9 Two-Minute Demo and Final Revision

9.1 Demo order

1. Problem bolo: search gives links, chatbot can give unsupported prose.
2. Ek focused example question select karo.
3. Four UI cards explain karo.
4. Research start karke progress sequence show karo.
5. Report mein [Sx] citations point out karo.
6. Sources tab mein same IDs and original URLs show karo.
7. Critic feedback and score show karo.
8. Search plan/Reader notes se pipeline transparency show karo.
9. Download and one follow-up question demonstrate karo.

9.2 Five lines yaad honi chahiye

1. "I preserved the tutorial's Search-Reader-Writer-Critic flow."
2. "Retrieval is deterministic; LLMs handle planning, comparison, writing and review."

3. "Gemini runs research and Groq optionally provides an independent Critic."
4. "Source IDs provide traceability, but I do not claim they guarantee truth."
5. "I kept the code sequential and simple, and I can name the production improvements I intentionally postponed."

9.3 Final checklist

- ☐ 60-second Hinglish and English pitch ready.
- ☐ Architecture without notes draw kar sakte ho.
- ☐ Har file ki responsibility explain kar sakte ho.
- ☐ Four stages ke inputs and outputs clear hain.
- ☐ Gemini/Groq rule and Tavily optionality clear hai.
- ☐ 413, PEP 668 and secrets story ready hai.
- ☐ Limitations honestly explain kar sakte ho.
- ☐ Demo query and deployed link test kiye hain.
- ☐ All exposed API keys rotated hain.

Final mindset

Interviewer ko buzzwords se impress karne ki jagah simple execution flow, real source grounding, design trade-offs aur honest limitations explain karo. Tumhe har abstraction defend karne ki zaroorat nahi padegi because code intentionally straight-forward hai.